Complete Guide: Monitoring Your Project with Prometheus

Overview

This guide shows you how to add Prometheus monitoring to your application in various programming languages.

---

1. Python Project Example (Flask/FastAPI)

Install Prometheus Client

```bash
pip install prometheus-client
```

Flask Application with Prometheus

```python
```

```python
# app.py
from flask import Flask, request
from prometheus_client import Counter, Histogram, Gauge, generate_latest, REGISTRY
import time
import psutil

app = Flask(__name__)

# Define Metrics
# Counter: Counts events (only goes up)
request_count = Counter(
    'app_requests_total',
    'Total number of requests',
    ['method', 'endpoint', 'status']
)

# Histogram: Measures distribution of values (like request duration)
request_duration = Histogram(
    'app_request_duration_seconds',
    'Request duration in seconds',
    ['method', 'endpoint']
)

# Gauge: Values that can go up and down (like CPU usage)
cpu_usage = Gauge('app_cpu_usage_percent', 'CPU usage percentage')
memory_usage = Gauge('app_memory_usage_bytes', 'Memory usage in bytes')
active_users = Gauge('app_active_users', 'Number of active users')

# Custom business metrics
orders_total = Counter('app_orders_total', 'Total orders placed', ['status'])
payment_amount = Histogram('app_payment_amount_dollars', 'Payment amounts')

# Middleware to track all requests
@app.before_request
def before_request():
    request.start_time = time.time()

@app.after_request
def after_request(response):
    request_duration_seconds = time.time() - request.start_time

    # Record metrics
    request_count.labels(
```

```
        method=request.method,
        endpoint=request.endpoint or 'unknown',
        status=response.status_code
    ).inc()

    request_duration.labels(
        method=request.method,
        endpoint=request.endpoint or 'unknown'
    ).observe(request_duration_seconds)

    return response

# Metrics endpoint for Prometheus to scrape
@app.route('/metrics')
def metrics():
    # Update system metrics before returning
    cpu_usage.set(psutil.cpu_percent())
    memory_usage.set(psutil.Process().memory_info().rss)

    return generate_latest(REGISTRY), 200, {'Content-Type': 'text/plain; charset=utf-8'}

# Example application routes
@app.route('/')
def home():
    return {'message': 'Hello World'}

@app.route('/api/users')
def get_users():
    # Simulate work
    time.sleep(0.1)
    return {'users': [{'id': 1, 'name': 'John'}]}

@app.route('/api/order', methods=['POST'])
def create_order():
    # Track business metrics
    order_status = request.json.get('status', 'pending')
    amount = request.json.get('amount', 0)

    orders_total.labels(status=order_status).inc()
    payment_amount.observe(amount)

    return {'order_id': 123, 'status': order_status}

@app.route('/health')
```

```
def health():
    return {'status': 'healthy'}

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

FastAPI Application with Prometheus

```python
```

```python
# fastapi_app.py
from fastapi import FastAPI, Request
from prometheus_client import Counter, Histogram, Gauge, generate_latest, REGISTRY
import time
import psutil

app = FastAPI()

# Define metrics
request_count = Counter(
    'fastapi_requests_total',
    'Total requests',
    ['method', 'endpoint', 'status']
)

request_duration = Histogram(
    'fastapi_request_duration_seconds',
    'Request duration',
    ['method', 'endpoint']
)

active_requests = Gauge('fastapi_active_requests', 'Active requests')

# Middleware
@app.middleware("http")
async def prometheus_middleware(request: Request, call_next):
    active_requests.inc()
    start_time = time.time()

    response = await call_next(request)

    duration = time.time() - start_time
    active_requests.dec()

    request_count.labels(
        method=request.method,
        endpoint=request.url.path,
        status=response.status_code
    ).inc()

    request_duration.labels(
        method=request.method,
        endpoint=request.url.path
```

```
    ).observe(duration)

    return response

@app.get("/metrics")
async def metrics():
    return generate_latest(REGISTRY)

@app.get("/")
async def root():
    return {"message": "FastAPI with Prometheus"}

@app.get("/api/data")
async def get_data():
    time.sleep(0.05)
    return {"data": [1, 2, 3]}
```

---

2. Node.js Project Example (Express)

Install Prometheus Client

```bash
npm install prom-client express
```

Express Application with Prometheus

```javascript
```

```
// server.js
const express = require('express');
const client = require('prom-client');

const app = express();
app.use(express.json());

// Create a Registry
const register = new client.Registry();

// Add default metrics (CPU, memory, etc.)
client.collectDefaultMetrics({ register });

// Custom Metrics
const httpRequestDuration = new client.Histogram({
  name: 'http_request_duration_seconds',
  help: 'Duration of HTTP requests in seconds',
  labelNames: ['method', 'route', 'status_code'],
  registers: [register]
});

const httpRequestTotal = new client.Counter({
  name: 'http_requests_total',
  help: 'Total number of HTTP requests',
  labelNames: ['method', 'route', 'status_code'],
  registers: [register]
});

const activeConnections = new client.Gauge({
  name: 'active_connections',
  help: 'Number of active connections',
  registers: [register]
});

const databaseQueryDuration = new client.Histogram({
  name: 'database_query_duration_seconds',
  help: 'Database query duration',
  labelNames: ['query_type'],
  registers: [register]
});

// Middleware to track metrics
app.use((req, res, next) => {
```

```
  const start = Date.now();
  activeConnections.inc();

  res.on('finish', () => {
    const duration = (Date.now() - start) / 1000;
    const route = req.route ? req.route.path : req.path;

    httpRequestDuration.observe(
      { method: req.method, route: route, status_code: res.statusCode },
      duration
    );

    httpRequestTotal.inc({
      method: req.method,
      route: route,
      status_code: res.statusCode
    });

    activeConnections.dec();
  });

  next();
});

// Metrics endpoint
app.get('/metrics', async (req, res) => {
  res.set('Content-Type', register.contentType);
  res.end(await register.metrics());
});

// Example routes
app.get('/', (req, res) => {
  res.json({ message: 'Hello from Node.js with Prometheus!' });
});

app.get('/api/users', (req, res) => {
  // Simulate database query
  const start = Date.now();

  setTimeout(() => {
    const duration = (Date.now() - start) / 1000;
    databaseQueryDuration.observe({ query_type: 'select' }, duration);

    res.json({ users: [{ id: 1, name: 'Alice' }] });
```

```
  }, 50);
});

app.post('/api/orders', (req, res) => {
  res.json({ order_id: 123, status: 'created' });
});

const PORT = 3000;
app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
  console.log(`Metrics available at http://localhost:${PORT}/metrics`);
});
```

---

3. Go Project Example

Install Prometheus Client

```bash
go get github.com/prometheus/client_golang/prometheus
go get github.com/prometheus/client_golang/prometheus/promhttp
```

Go Application with Prometheus

```go
```

```go
// main.go
package main

import (
    "fmt"
    "log"
    "net/http"
    "time"

    "github.com/prometheus/client_golang/prometheus"
    "github.com/prometheus/client_golang/prometheus/promhttp"
)

var (
    httpRequestsTotal = prometheus.NewCounterVec(
        prometheus.CounterOpts{
            Name: "http_requests_total",
            Help: "Total number of HTTP requests",
        },
        []string{"method", "endpoint", "status"},
    )

    httpRequestDuration = prometheus.NewHistogramVec(
        prometheus.HistogramOpts{
            Name:    "http_request_duration_seconds",
            Help:    "Duration of HTTP requests in seconds",
            Buckets: prometheus.DefBuckets,
        },
        []string{"method", "endpoint"},
    )

    activeConnections = prometheus.NewGauge(
        prometheus.GaugeOpts{
            Name: "active_connections",
            Help: "Number of active connections",
        },
    )
)

func init() {
    // Register metrics with Prometheus
    prometheus.MustRegister(httpRequestsTotal)
    prometheus.MustRegister(httpRequestDuration)
```

```go
    prometheus.MustRegister(activeConnections)
}

// Middleware to track metrics
func prometheusMiddleware(next http.HandlerFunc) http.HandlerFunc {
    return func(w http.ResponseWriter, r *http.Request) {
        start := time.Now()
        activeConnections.Inc()
        defer activeConnections.Dec()

        // Create a custom response writer to capture status code
        wrappedWriter := &responseWriter{ResponseWriter: w, statusCode: http.StatusOK}

        next(wrappedWriter, r)

        duration := time.Since(start).Seconds()

        httpRequestsTotal.WithLabelValues(
            r.Method,
            r.URL.Path,
            fmt.Sprintf("%d", wrappedWriter.statusCode),
        ).Inc()

        httpRequestDuration.WithLabelValues(
            r.Method,
            r.URL.Path,
        ).Observe(duration)
    }
}

type responseWriter struct {
    http.ResponseWriter
    statusCode int
}

func (rw *responseWriter) WriteHeader(code int) {
    rw.statusCode = code
    rw.ResponseWriter.WriteHeader(code)
}

func homeHandler(w http.ResponseWriter, r *http.Request) {
    w.Header().Set("Content-Type", "application/json")
    fmt.Fprintf(w, `{"message": "Hello from Go with Prometheus!"}`)
}
```

```go
func apiHandler(w http.ResponseWriter, r *http.Request) {
    time.Sleep(50 * time.Millisecond) // Simulate work
    w.Header().Set("Content-Type", "application/json")
    fmt.Fprintf(w, `{"data": [1, 2, 3]}`)
}

func main() {
    // Wrap handlers with Prometheus middleware
    http.HandleFunc("/", prometheusMiddleware(homeHandler))
    http.HandleFunc("/api/data", prometheusMiddleware(apiHandler))

    // Expose metrics endpoint
    http.Handle("/metrics", promhttp.Handler())

    fmt.Println("Server starting on :8080")
    fmt.Println("Metrics available at http://localhost:8080/metrics")
    log.Fatal(http.ListenAndServe(":8080", nil))
}
```

---

4. Configure Prometheus to Scrape Your Application

Update Prometheus Configuration

```bash
sudo nano /etc/prometheus/prometheus.yml
```

Add your application as a target:

```yaml
```

```yaml
scrape_configs:
  - job_name: 'prometheus'
    static_configs:
      - targets: ['localhost:9090']

  - job_name: 'node_exporter'
    static_configs:
      - targets: ['localhost:9100']

  # Python Flask/FastAPI app
  - job_name: 'python_app'
    scrape_interval: 5s
    static_configs:
      - targets: ['localhost:5000']
    metrics_path: '/metrics'

  # Node.js Express app
  - job_name: 'nodejs_app'
    scrape_interval: 5s
    static_configs:
      - targets: ['localhost:3000']
    metrics_path: '/metrics'

  # Go application
  - job_name: 'go_app'
    scrape_interval: 5s
    static_configs:
      - targets: ['localhost:8080']
    metrics_path: '/metrics'
```

Restart Prometheus

```bash
sudo systemctl restart prometheus
```

---

5. Verify Everything is Working

Check Targets

1. Open Prometheus UI: `http://your-ip:9090`
2. Go to **Status → Targets**

3. All your applications should show as "UP"

Query Your Metrics

Go to **Graph** tab and try these queries:

```promql
# Request rate
rate(app_requests_total[5m])

# Average request duration
rate(app_request_duration_seconds_sum[5m]) / rate(app_request_duration_seconds_count[5m])

# CPU usage
app_cpu_usage_percent

# Active connections
active_connections

# 95th percentile response time
histogram_quantile(0.95, rate(http_request_duration_seconds_bucket[5m]))
```

---

6. Create Systemd Service for Your App

Python Flask Example

```bash
sudo nano /etc/systemd/system/myapp.service
```

```ini
```

```
[Unit]
Description=My Python Flask App
After=network.target

[Service]
User=ubuntu
WorkingDirectory=/home/ubuntu/myapp
Environment="PATH=/home/ubuntu/myapp/venv/bin"
ExecStart=/home/ubuntu/myapp/venv/bin/python app.py
Restart=always

[Install]
WantedBy=multi-user.target
```

Start the service:

```bash
sudo systemctl daemon-reload
sudo systemctl start myapp
sudo systemctl enable myapp
```

---

7. Common Metric Types Explained

Counter

- Only increases (never decreases)
- Use for: total requests, errors, orders placed

```python
requests_total.inc()  # Increment by 1
errors_total.labels(error_type='500').inc()
```

Gauge

- Can go up or down
- Use for: current users, temperature, queue size

```python
```

```
active_users.set(100)
active_users.inc()  # Increase by 1
active_users.dec()  # Decrease by 1
```

Histogram

- Samples observations and counts them in buckets
- Use for: request duration, response size

```python
request_duration.observe(0.5)  # 0.5 seconds
```

Summary

- Similar to histogram but calculates quantiles
- Use for: request durations, payload sizes

---

8. Best Practices

1. **Naming Convention**
   - Use `_total` suffix for counters
   - Use `_seconds` for durations
   - Use `_bytes` for sizes
   - Example: `http_requests_total`, `request_duration_seconds`
2. **Labels**
   - Don't use high cardinality labels (user IDs, timestamps)
   - Good labels: `status_code`, `method`, `endpoint`
   - Bad labels: `user_id`, `timestamp`, `session_id`
3. **Scrape Interval**
   - Default: 15s
   - For critical apps: 5s
   - For less critical: 30s-60s
4. **Metrics Endpoint**

- Always use `/metrics`
- Return metrics in Prometheus format
- Should be fast (< 100ms)

---

9. Testing Your Metrics

Generate Traffic

```bash
# Use Apache Bench
ab -n 1000 -c 10 http://localhost:5000/

# Or curl in a loop
for i in {1..100}; do curl http://localhost:5000/; done
```

Check Metrics Endpoint

```bash
curl http://localhost:5000/metrics
```

You should see output like:

```
# HELP app_requests_total Total number of requests
# TYPE app_requests_total counter
app_requests_total{endpoint="/",method="GET",status="200"} 42.0
```

Next Steps

1. **Install Grafana** for beautiful dashboards
2. **Set up Alerting** for critical metrics
3. **Add more exporters** (MySQL, Redis, etc.)
4. **Create custom dashboards**
5. **Set up retention policies**

Your application is now being monitored by Prometheus! 🎉